Mechanical Engineering Skills in Claude

Yayati Jadhav Peter Pak J. Barkley Akshay Badagabettu E. George
Aakash S. Oza Parshin Shojaee Z. Wang Achuth Chandrasekhar
Francis Ogoke Parand Akbari Amir Barati Farimani*

Department of Mechanical Engineering, Carnegie Mellon University, Pittsburgh, PA 15213, USA

Abstract

Large language models can now plan multi-step engineering work, call software and read the results, yet they remain unreliable at the parts of mechanical engineering that matter most: getting the numbers right, choosing the right check, and knowing when a result is outside the range where a method holds. We present *Mechanical Engineering Skills*, an open library of 32 agent skills for design, analysis and manufacturing that run in Claude (Claude Code and Claude.ai) and in other agents that read the open Agent Skills format. Each skill is a folder that pairs concise operating instructions with tested scripts, so the model orchestrates while deterministic tools compute, and every result is returned with the check that supports it. Nineteen skills package published or preprint research from our laboratory, spanning text-to-CAD, linkage synthesis, CFD case setup, stress-field and lattice surrogates, equation discovery, metal and polymer additive-manufacturing process planning, and in-process monitoring. We describe the design principles, the procedure we used to turn a paper into a reliable skill, and what the process revealed: reproductions that match the source papers (rocket apogee within 4% of OpenRocket, exact homogenization on solvable cases, process suggestions that match operators' choices batch by batch) and also unit errors in released data, train/test leakage that inflates reported accuracy, and under-specified details that block reproduction. We argue that skills are a practical unit for transferring engineering research into everyday agent use, and that writing a skill is itself a strong test of whether a method is reproducible. The library is available at https://github.com/BaratiLab/mechanical-engineering-skills.

1 Introduction

Mechanical engineering work is a chain of small, checkable steps: read a requirement, pick a model, compute, compare against a hand estimate or a known case, and decide. General-purpose language-model agents are now capable of driving such chains: they write CAD scripts, set up simulations, and read sensor data. Our own work has shown this across the life cycle of a part, from text-to-CAD [1, 2] and mechanism synthesis [3] to LLM-in-the-loop structural design [4], autonomous CFD [5], additive-manufacturing process planning [6–8] and print monitoring [9].

Two obstacles keep this research from becoming everyday practice. First, each method lives in its own code base, with its own environment, data and conventions, and is rarely packaged for someone who simply has an engineering question. Second, agents left to themselves make the kind of errors engineers cannot accept: arithmetic slips, wrong units, a correlation used outside its range, or a confident answer with no evidence behind it.

Agent Skills [10, 11] offer a light-weight remedy. A skill is a directory with a `SKILL.md` file—a short description that tells the agent when to use it, followed by instructions—plus any scripts

*Corresponding author: barati@cmu.edu

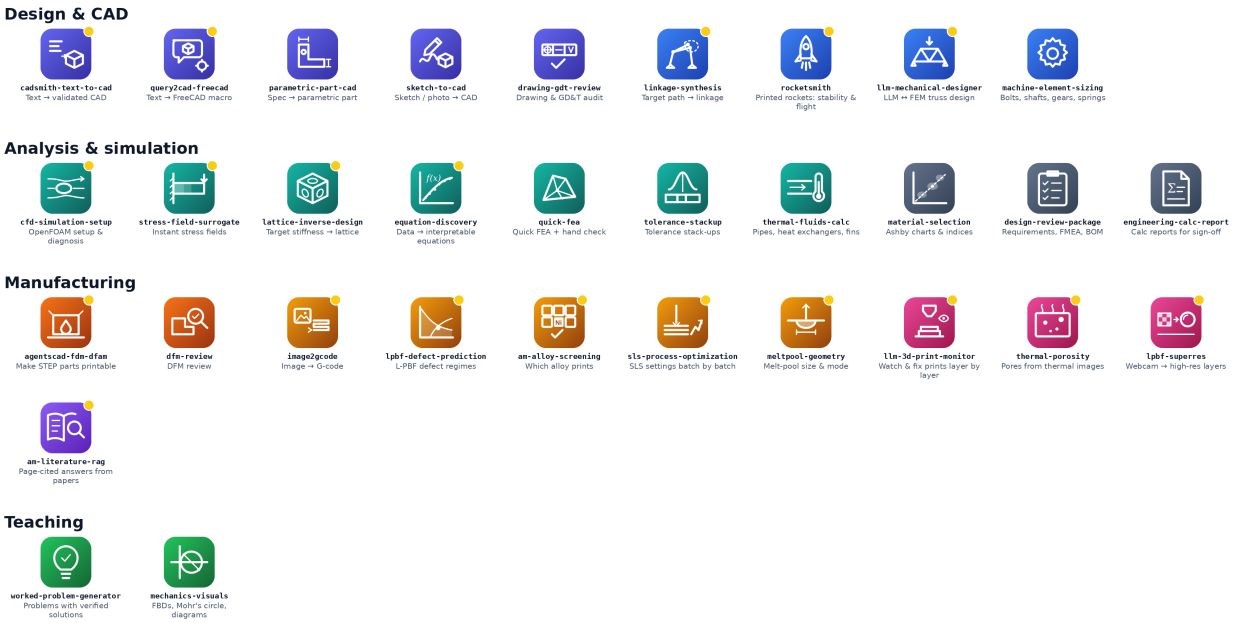


Figure 1: The 32 skills in Mechanical Engineering Skills, grouped by area. Tile colour encodes the sub-area; a gold dot marks the 19 skills that package a Barati Farimani Lab paper. Each skill is installed with `npx skills add BaratiLab/mechanical-engineering-skills`.

and reference files the instructions point to. The agent sees only the one-line descriptions until a task matches, then loads the instructions, and reads deeper files or runs scripts only when needed (*progressive disclosure*). Skills are portable across Claude Code, Claude.ai and other agents that adopted the format, and they are installed with a single command.

This paper presents *Mechanical Engineering Skills* [12], a library of 32 such skills (Fig. 1). Our contributions are:

1. **A library** covering design, analysis and manufacturing, in which 19 skills package peer-reviewed or preprint research from our laboratory and 13 cover core engineering tasks (machine-element sizing, FEA with hand checks, tolerance stack-ups, material selection, design reviews and teaching aids).
2. **Design principles** for engineering skills: tools compute and the model orchestrates; every number carries its check; the paper's protocol is reproduced but its weaknesses are corrected and declared; and the skill states the boundary of its validity.
3. **A procedure for turning a paper into a skill**, and a report of what it uncovered when applied to 19 papers, including reproduced results, errors in released data, evaluation leakage and missing details.

2 Skills as a unit of engineering capability

Figure 2 shows the structure we use for every skill. The `SKILL.md` front matter carries a description written for the agent's routing decision: what the skill does, what it is based on, and the situations that should trigger it. The body is a short workflow—typically four to six numbered steps—with the commands to run and, crucially, the *judgement calls* an experienced engineer would make: which

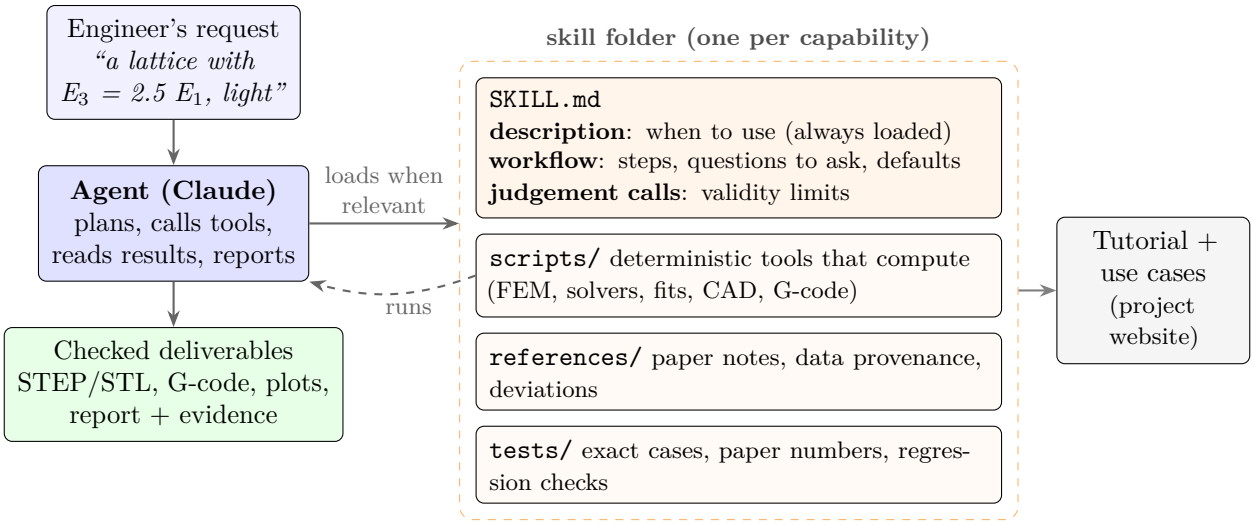


Figure 2: Anatomy of an engineering skill and how an agent uses it. Only the one-line description is always in the agent's context; instructions, scripts and references are loaded on demand. Tests and a public tutorial accompany every research skill.

inputs to ask for, what to report alongside the headline number, and where the method stops being valid. Long material (paper notes, provenance, interpretation choices) sits in `references/` so that it costs no context unless needed.

The `scripts/` directory holds the computation. Scripts are ordinary command-line Python programs with JSON output so that the agent can parse results reliably. Examples include a periodic voxel homogenizer for lattices, a plane-stress Q4 finite-element solver, a Barrowman stability and one-dimensional flight simulator for rockets, a Rosenthal melt-pool solver, an SQLite build log with Gaussian-process suggestions for selective laser sintering, and validators for OpenFOAM cases. Each research skill also ships a test suite (`tests/`) and a tutorial on the project website that walks through the paper's main experiment with real outputs.

3 Design principles

P1. The model orchestrates; tools compute. Every number a user might act on is produced by code, never by the model's arithmetic. RocketSmith [13] reached the same conclusion after flying four agent-designed rockets; we apply it library-wide. The model remains responsible for what it is good at: eliciting requirements, choosing a method, chaining tools, and explaining results.

P2. Every result carries its check. Skills return the evidence for a result alongside it: a hand calculation for an FEA run, the exact laminate solution before trusting a lattice homogenization, re-simulation of a surrogate's best candidate, a kernel validity and volume check after a CAD edit. Where a published method used a learned scorer for acceptance (e.g. a surrogate predicting a generated lattice's modulus), the skill uses the physics model for acceptance and keeps the learned model for guidance.

P3. Reproduce the paper, then fix what would mislead a user. Each research skill implements the paper's protocol so that its headline numbers can be reproduced, and then changes

defaults where the original choice would mislead a practitioner—most often the evaluation split (§7). Every deviation is listed in the skill's paper notes with the reason and the evidence.

P4. State the boundary. Skills say what they do not do: elastic homogenization is not crash performance; a flight simulation is a design check, not a launch approval; a defect classifier trained on one machine family is not a process qualification. These statements live in the instructions so the agent repeats them at the point of use.

P5. Ask once, then act. Instructions list the few inputs that change the answer (a material's printed modulus, minimum wall thickness, certification level) and sensible defaults for the rest, so that the agent asks one focused question rather than many generic ones.

4 The library

Table 1 lists the 19 research skills with the source paper and the checks each one runs. The 13 general skills cover machine-element sizing with safety factors, linear static and modal FEA with a mandatory hand-calculation comparison, tolerance stack-ups (worst case, RSS and Monte Carlo), thermal-fluids calculations with CoolProp properties, Ashby-index material selection [14], design-review packages (requirements, FMEA, BOM), calculation reports, drawing and GD&T review, parametric and sketch-to-CAD modelling, DFM review, and two teaching skills that generate problems with code-verified solutions and mechanics diagrams.

5 From paper to skill

We used the same procedure for every research skill:

1. **Extract the engineering workflow**, not the model. Most papers describe a model and a benchmark; a practitioner needs the steps around it—what to measure, how to prepare inputs, how to decide. We write those steps first.
2. **Re-derive the physics and build exact checks** before any learned component: a solid cell and a laminate for homogenization, Euler–Bernoulli bending for a stress generator, published OpenRocket predictions for flight.
3. **Reproduce the paper's headline experiment** with its protocol, and record the numbers in the tutorial.
4. **Stress-test the protocol**: change the split to unseen geometries, parts or studies; sanitize units; replay sequential decisions without future information.
5. **Write the instructions last**, including the judgement calls discovered in steps 2–4, and add tests for every check.

When code or data were unavailable or impractical (several repositories were placeholders, one was 13 GB), the skill implements the method from the paper and ships a generator or rehearsal data so that the workflow can be exercised end to end.

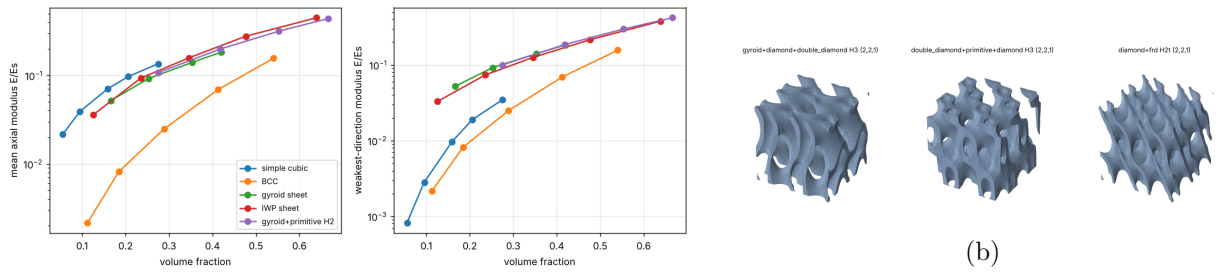
Table 1: Research skills, their source papers and the checks they run. Area: D = design and CAD, A = analysis and simulation, M = manufacturing.

Skill	Area	Source	What the skill computes and checks
`cadsmith-text-to-cad`	D	[1]	CadQuery generation with kernel measurements, three-view judge; Chamfer/F1/IoU after ICP
`query2cad-freecad`	D	[2]	FreeCAD macros, error repair, VQA scoring, caption-driven refinement
`linkage-synthesis`	D	[3]	Node-by-node linkage construction, dyadic and kinematic validation, Chamfer to target path
`llm-mechanical-designer`	D	[4]	LLM-proposed trusses evaluated by exact FEM, ranked history
`rocketsmith`	D	[13]	Barrowman CP [15], stability, flight simulation, fin sizing, C_D calibration, printable CAD
`cfd-simulation-setup`	A	[5]	OpenFOAM case building; acceptance only by registered physics contracts
`stress-field-surrogate`	A	[16]	Q4 FEM data generator (beam theory within 1%), diffusion and regression surrogates
`lattice-inverse-design`	A	[17]	TPMS/hybrid cells, printability, periodic homogenization (exact on laminates), search, 3D diffusion
`equation-discovery`	A	[18]	LLM-proposed equation skeletons, sandboxed BFGS fits, in- and out-of-domain NMSE
`agentscad-fdm-dfam`	M	[19]	Overhang analysis, reorientation, teardrops and chamfers, kernel-verified STEP
`image2gcode`	M	[20]	Image to toolpath and G-code; area check against the drawn part
`lpbf-defect-prediction`	M	[6]	Keyhole/lack-of-fusion/balling regimes and process maps, grouped evaluation
`am-alloy-screening`	M	[7]	Thermo-Calc or flagged estimates, Rosenthal LoF maps, unit sanitation, ranking
`sls-process-optimization`	M	[8]	Build memory, ASTM D638/D790 reduction, bounded Bayesian next batch
`meltpool-geometry`	M	[21]	Rosenthal [22] and power laws, cross-study validation, own-track calibration
`llm-3d-print-monitor`	M	[9]	Layer images, bounded G-code corrections via Klipper/Moonraker, mock printer
`thermal-porosity`	M	[23]	Pyrometry to pore count and porosity map; CT alignment
`lpbf-superres`	M	[24]	Diffusion super-resolution of layer images, registration, edge roughness
`am-literature-rag`	M	[25]	Page-cited retrieval over the user's own literature; retrieval evaluation

6 Case studies

6.1 Lattice inverse design (GLU3D)

GLU3D [17] generates lattice unit cells from a target stiffness with a 3D conditional diffusion model trained on homogenized TPMS and hybrid-TPMS cells. The skill first builds the physics: implicit cells, printability filters (one periodic solid that reaches every face, no sealed cavities), and an energy-based periodic voxel homogenizer [26] that returns the full 6×6 stiffness. The homogenizer reproduces a solid cell exactly and matches the exact C_{33}, C_{44} and C_{11} of a two-phase laminate to 10^{-6}; a sheet gyroid at volume fraction 0.25 gives $E/E_s = 0.086$ with exact cubic symmetry.



(a) (b)

Figure 3: Lattice inverse design. (a) Mean axial and weakest-direction modulus against volume fraction for simple cubic, BCC, sheet gyroid, sheet I-WP and a gyroid–primitive hybrid, from the skill's homogenizer. (b) The three best cells returned by search for $E_1 = E_2 = 0.06$, $E_3 = 0.15$; anisotropy comes from two periods along x and y.

Figure 3(a) compares stiffness against simple-cubic and BCC strut lattices: at equal density the TPMS and hybrid sheets are about 5× stiffer than BCC and 3× stiffer than simple cubic in its weakest direction, though simple cubic is 1.3× stiffer along its own axes—which is why the skill reports the directional minimum.

For one-off designs the skill searches a homogenized dataset and refines thickness and level-set bias. For an anisotropic target $E_1 = E_2 = 0.06$, $E_3 = 0.15$ (relative to the solid) it returned a gyroid–diamond–double-diamond hybrid with $E = (0.057, 0.058, 0.151)$ at volume fraction 0.26 after re-checking at 48^3 (Fig. 3b), in two minutes on a laptop CPU.

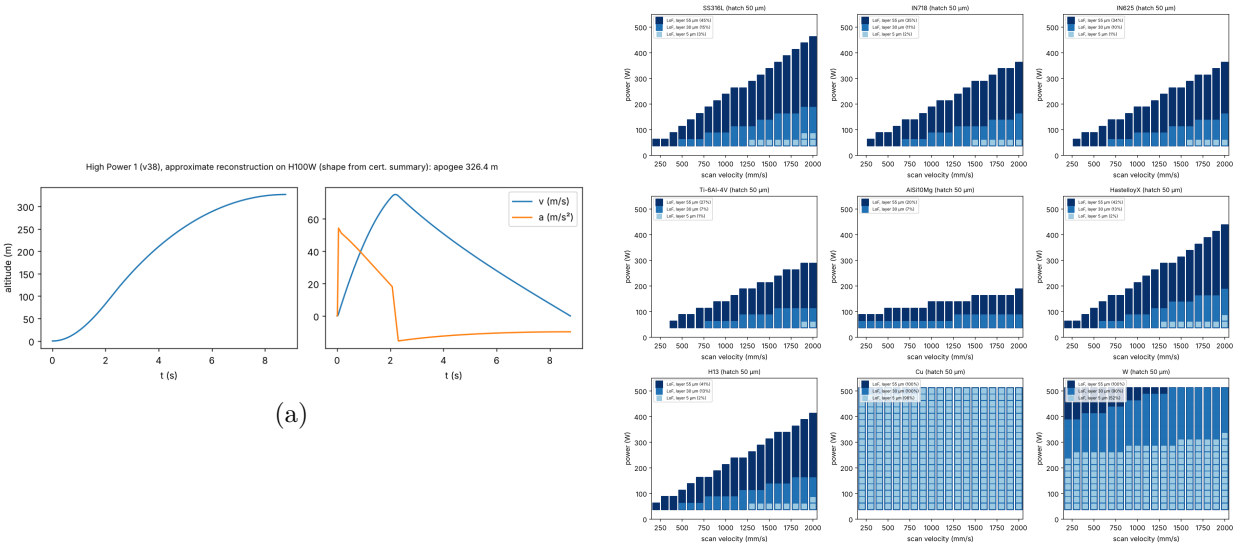
The generative route reproduces the paper's training recipe. A literal reproduction on a small dataset (300 cells, 16^3, CPU) produced unstructured fields; four changes produced structured samples: training on the periodic grid with circular padding (which makes every generated cell exactly periodic), a sinusoidal-plus-MLP time embedding in place of a per-step lookup table, scaling the conditioning vector inside the encoder, and a longer learning-rate plateau patience. Stiffness error after re-homogenization remained 200–280% at this data scale, against 3723 cells at 64^3 and four GPUs in the paper; the skill therefore recommends search for production designs and accepts generated cells only after re-homogenization.

6.2 Printed rockets (RocketSmith)

RocketSmith [13] designed four FDM-printed Class 2 rockets that all flew stably; altimeter apogees were 80–84% of predictions. The skill reconstructs two of them from the paper and predicts apogee and speed within a few percent of OpenRocket: 326 m and 75 m/s against 338.8 m and 76.4 m/s for HP1, and 568 m and 113.7 m/s against 570.1 m and 115.2 m/s for HP4 (Fig. 4a). Fitting the drag coefficient to the measured apogees (276 m and 479 m) gives $C_D \approx 0.84$ and 0.76, much higher than the 0.4–0.5 typical of smooth commercial airframes—a direct, actionable explanation for the shortfall: printed surfaces and fin geometry. The skill's fin-sizing loop also flags that HP1's fins (95 mm span) give marginal stability and computes the 162 mm span needed for 1.26 calibers.

6.3 Alloy screening and the cost of unchecked units

The alloy-evaluation paper [7] asks an agent to judge whether a known or new alloy prints by laser powder-bed fusion, from Thermo-Calc properties and Rosenthal lack-of-fusion maps. The skill reproduces the paper's ranking (IN625 above SS316L, copper last; Fig. 4b). While porting the released property files we found densities about 1000× too high and heat capacities in $\mathrm{J\,mol^{-1}\,K^{-1}}$



(a) (b)

Figure 4: (a) Simulated flight of the reconstructed HP1 rocket. (b) Lack-of-fusion process maps for known alloys produced by `am-alloy-screening` after unit sanitation.

instead of $\mathrm{J\,kg^{-1}\,K^{-1}}$, which shifts thermal diffusivity by a factor of 50–100. The skill now sanitizes units on load and reports what it corrected, and its outputs are explicitly keyed, since the paper traced an LLM error to an unlabeled list of tuples.

6.4 Sequential process optimization (SLS)

For polymer selective laser sintering [8], the skill keeps a build log of 21 published batches across three materials, reduces ASTM D638/D790 tests, and suggests the next batch with a bounded Gaussian-process expected-improvement step. To test it fairly we replayed the campaign, showing the suggester only the batches available at each point. After batches A–C it proposed raising hotspot overlap to 50%—what batch D did; after A–E it proposed fill energy 28 mJ/mm—batch I; after A–G, 30 mJ/mm. The suggestions are bounded to small steps and refuse surface temperatures within 3 °C of the melt.

6.5 Equation discovery (LLM-SR)

LLM-SR [18] lets a model propose equation skeletons as code while an optimizer fits constants. The skill runs proposals in an isolated interpreter after static safety checks, fits with BFGS from multiple starts, and scores in-domain and out-of-domain splits. On the stress–strain benchmark the best discovered skeleton reached normalized MSE 0.021/0.025/0.074 (train/in-domain/out-of-domain) against 0.161/0.150/0.680 for a cubic polynomial and 0.38 for linear regression; the advantage is largest where it matters for engineering, out of domain.

7 What writing skills revealed

Evaluation leakage is common and large. Several papers used random splits in which the same geometry, melt pool or study appears in training and test data. Re-evaluating on held-out

groups changed the conclusions a user would draw (Table 2). Each affected skill defaults to the grouped split and reports both numbers.

Table 2: Paper protocol versus the grouped split each skill uses by default.

Skill	Metric	Random split	Grouped split
`lpbf-defect-prediction`	regime accuracy	98.1%	87.9% (melt pools held out)
`stress-field-surrogate`	MAE (MPa), equal training	0.57	0.81 (geometries held out)
`meltpool-geometry`	mode accuracy	82.6%	≈45% (studies held out)

Released data can be wrong in ways an agent will not notice. The unit errors in the alloy data would have produced plausible-looking process maps. Agents need explicit sanity rules (order-of-magnitude bounds on density, heat capacity and conductivity), which the skill encodes.

Papers under-specify what reproduction needs. Missing supporting information (the lattice families), ambiguous signs in a scaling equation, and training details whose literal reading fails on smaller data (a per-step time-embedding table; a plateau scheduler that anneals the learning rate to nothing within 60 epochs) each blocked or degraded reproduction. Writing them down in the skill notes turns tacit knowledge into something the next user, human or agent, can act on.

Small physics models are worth more than they look. Calibrating one drag coefficient from a measured apogee, or one melt-pool constant from a few single tracks, closed most of the gap between prediction and measurement. Skills therefore expose calibration as a first-class step.

8 Limitations and future work

The library has been validated by tests and by reproducing published results, not yet by a controlled study of engineers using it; we plan task-based evaluations comparing agents with and without skills on design, analysis and manufacturing problems. Several skills need external software (OpenFOAM, FreeCAD, Thermo-Calc) or hardware (a Klipper printer, a powder-bed machine); they ship validators, mock devices or rehearsal data, but real use requires the tools. Learned components were rehearsed at small scale on CPU; their headline accuracy depends on retraining at the papers' scale. Finally, skills inherit the limits of their methods; the instructions state them, but they cannot replace an engineer's sign-off on safety-critical decisions.

9 Conclusion

Mechanical Engineering Skills packages 32 engineering capabilities, 19 of them from our research, in a form that a general agent can discover, load and use correctly. The central idea is simple: let the model plan and explain, let tested code compute, and return every result with its check. Applying this discipline to our own papers reproduced their key results and also exposed data errors, leakage and missing details—suggesting that a skill is not only a delivery format but a useful standard for research reproducibility. The library is open-source under the MIT license; we invite contributions of skills from other groups.

Availability. Code, tests and tutorials:
https://github.com/BaratiLab/mechanical-engineering-skills;
website: https://baratilab.github.io/mechanical-engineering-skills/.
Install all skills with `npx skills add BaratiLab/mechanical-engineering-skills -all`.

References

[1] J. Barkley, R. Loghmani, and A. Barati Farimani. CADSmith: Multi-agent CAD generation with programmatic geometric validation. *arXiv preprint arXiv:2603.26512*, 2026.

[2] A. Badagabettu, S. S. Yarlagadda, and A. Barati Farimani. Query2CAD: Generating CAD models using natural language queries. *arXiv preprint arXiv:2406.00144*, 2024.

[3] Y. Jadhav and A. Barati Farimani. LinkD: AutoRegressive diffusion model for mechanical linkage synthesis. *arXiv preprint arXiv:2601.04054*, 2026.

[4] Y. Jadhav and A. Barati Farimani. Large language model agent as a mechanical designer. *Journal of Engineering Design*, 2026. arXiv:2404.17525.

[5] A. S. Oza, R. F. Johnson, and A. Barati Farimani. CFD Forge: A physics-constrained agentic framework for autonomous CFD. *arXiv preprint arXiv:2610.14104*, 2026.

[6] P. Pak and A. Barati Farimani. AdditiveLLM: Large language models predict defects in metals additive manufacturing. *Additive Manufacturing Letters*, 14:100292, 2025. doi: 10.1016/j.addlet.2025.100292.

[7] P. Pak, A. Chandrasekhar, and A. Barati Farimani. Agentic additive manufacturing alloy evaluation. *Additive Manufacturing Letters*, 17:100355, 2026. doi: 10.1016/j.addlet.2026.100355.

[8] P. Pak, V. Alvarado, and A. Barati Farimani. AI agentic selective laser sintering process optimization. *arXiv preprint arXiv:2608.25928*, 2026.

[9] Y. Jadhav, P. Pak, and A. Barati Farimani. LLM-3D Print: Large language models to monitor and control 3D printing. *Additive Manufacturing*, 114:105027, 2025. arXiv:2408.14307.

[10] Anthropic. Equipping agents for the real world with Agent Skills. Anthropic Engineering blog, October 2025. URL https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills.

[11] Agent Skills. Agent skills: an open format for packaging agent capabilities, 2025. URL https://agentskills.io. Published as an open standard in December 2025.

[12] Barati Farimani Lab. Mechanical engineering skills. GitHub repository, 2026. URL https://github.com/BaratiLab/mechanical-engineering-skills.

[13] P. Pak, J. Barkley, R. Loghmani, D. Baich, A. Pamal, and A. Barati Farimani. RocketSmith: Agentic additive manufacturing of high-powered rockets. *arXiv preprint arXiv:2606.00097*, 2026.

[14] M. F. Ashby. *Materials Selection in Mechanical Design*. Butterworth-Heinemann, 4 edition, 2011.

[15] J. S. Barrowman. The practical calculation of the aerodynamic characteristics of slender finned vehicles. M.s. thesis, The Catholic University of America, 1967.

[16] Y. Jadhav, J. Berthel, C. Hu, R. Panat, J. Beuth, and A. Barati Farimani. StressD: 2D stress estimation using denoising diffusion model. *Computer Methods in Applied Mechanics and Engineering*, 416:116343, 2023. doi: 10.1016/j.cma.2023.116343.

[17] Y. Jadhav, J. Berthel, C. Hu, R. Panat, J. Beuth, and A. Barati Farimani. Generative lattice units with 3D diffusion for inverse design: GLU3D. *Advanced Functional Materials*, 34:2404165, 2024. doi: 10.1002/adfm.202404165.

[18] P. Shojaee, K. Meidani, S. Gupta, A. Barati Farimani, and C. K. Reddy. LLM-SR: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations (ICLR)*, 2025. Oral; arXiv:2404.18400.

[19] E. George, C. Keefe, P. Pak, and A. Barati Farimani. AgentsCAD: Automated design for manufacturing of FDM parts via multi-agent LLM reasoning and geometric feature recognition. *arXiv preprint arXiv:2607.02448*, 2026.

[20] Z. Wang, Y. Jadhav, P. Pak, and A. Barati Farimani. Image2Gcode: Image-to-G-code generation for additive manufacturing using diffusion-transformer model. *Additive Manufacturing Letters*, 17:100374, 2026. doi: 10.1016/j.addlet.2026.100374.

[21] P. Akbari, F. Ogoke, N.-Y. Kao, K. Meidani, C.-Y. Yeh, W. Lee, and A. Barati Farimani. MeltpoolNet: Melt pool characteristic prediction in metal additive manufacturing using machine learning. *Additive Manufacturing*, 55:102817, 2022. doi: 10.1016/j.addma.2022.102817.

[22] D. Rosenthal. The theory of moving sources of heat and its application to metal treatments. *Transactions of the American Society of Mechanical Engineers*, 68:849–866, 1946.

[23] P. Pak, F. Ogoke, A. Polonsky, A. Garland, D. S. Bolintineanu, D. R. Moser, M. Arnhart, J. Madison, T. Ivanoff, J. Mitchell, B. Jared, B. Salzbrenner, M. J. Heiden, and A. Barati Farimani. ThermoPore: Predicting part porosity based on thermal images using deep learning. *Additive Manufacturing*, page 104503, 2024. doi: 10.1016/j.addma.2024.104503.

[24] F. Ogoke, S. Kalambettu Suresh, J. Adamczyk, D. Bolintineanu, A. Garland, M. Heiden, and A. Barati Farimani. Deep learning based optical image super-resolution via generative diffusion models for layerwise in-situ LPBF monitoring. *Additive Manufacturing*, 107:104790, 2025. doi: 10.1016/j.addma.2025.104790.

[25] A. Chandrasekhar, J. Chan, F. Ogoke, O. Ajenifujah, and A. Barati Farimani. AMGPT: A large language model for contextual querying in additive manufacturing. *Additive Manufacturing Letters*, 11:100232, 2024. doi: 10.1016/j.addlet.2024.100232.

[26] G. Dong, Y. Tang, and Y. F. Zhao. A 149 line homogenization code for three-dimensional cellular materials written in MATLAB. *Journal of Engineering Materials and Technology*, 141 (1):011005, 2019.